Additional Topics to Explore Next

We can't cover every single aspect of Sidekiq or background jobs in this book. While we've gone through common and critical aspects of running Sidekiq, there are a few other areas you may wish to explore or know about.

Request-Scoped Data Is Available to Your Jobs

The Rails `CurrentAttributes` class allows you to store request-scoped data in a common location available to all classes.[27] For example, you could put the currently logged-in user's ID into `CurrentAttributes`. Since `CurrentAttributes` uses memory, it may seem that when you queue a job, that memory won't be available when the job is executed.

It's true that the memory used by `CurrentAttributes` won't be available, but Sidekiq can preserve this for you. Suppose you have created the class `Current` in `app/models/current.rb`, as documented by Rails' API. You can arrange to have Sidekiq store everything in the job payload, and restore it before the job executes. Put code like this in `config/initializers/sidekiq.rb`:

```
require "sidekiq/middleware/current_attributes"
Sidekiq::CurrentAttributes.persist(Current)
```

Queue Many Jobs at Once with `perform_bulk`

You may find yourself needing to queue many jobs at once. For example, suppose instead of queuing `RequestOrderFulfillmentJob` right after sending email, you wanted to request fulfillment for all orders placed that day at midnight, all in one group. While you could iterate over each unfulfilled order via `Order.where(fulfillment_request_id: nil).each`, this would make many Redis calls and could perform poorly as your database gets larger.

Instead, you can use `perform_bulk`. It expects an array of job arguments (essentially, an array of arrays), and a `batch_size:` to allow Sidekiq to batch